

Software Controller Proposal for SGLang

How agent-workflow knowledge can improve deadlines, GPU goodput, and KV-cache placement - and how to determine whether any fast path should later become hardware.

Scope: single SGLang worker first; multi-worker routing is deliberately deferred.

Research and implementation blueprint | 14 September 2026

Central thesis

The harness knows what the request means and what may happen next. SGLang knows what the request costs right now. The controller joins those two views and makes bounded resource decisions.

Contents

1. Executive summary	5
2. Why harness knowledge changes the problem	5
2.1 Knowledge the harness already has	6
2.2 Knowledge SGLang already has	6
3. The signal contract: what the harness should communicate	7
3.1 Core signal groups	7
3.2 Minimal first-version schema	8
3.3 Signals must include uncertainty	8
4. What the backend can do with semantic knowledge	8
4.1 Deadline-aware admission and scheduling	8
4.2 Proactive KV lifecycle management	9
4.3 Better batching and prefix locality	9
4.4 Work elimination and load shedding	9
4.5 Fairness, isolation and observability	9
5. How one SGLang worker schedules requests	9
5.1 Request path	9
5.2 The important queues and states	10
5.3 Decisions the scheduler makes	10
5.4 Native policy and capacity mechanisms	11
6. What HiCache provides - and what it does not	11
7. The five tool-wait actions, one by one	12
7.1 Correct prefetch calculation	12
7.2 Enforcement state machine	13

8. The three core hooks for a single worker	13
Hook 1 - Request intent and lifecycle events	13
Hook 2 - Scheduler policy/admission hook	13
Hook 3 - KV lifecycle hook	14
9. Concrete software architecture	14
9.1 Components	14
9.2 Control cadence	14
9.3 Recommended decision rule	15
10. Detailed single-worker operating example	15
10.1 Timeline	15
10.2 Possible failure outcomes	16
11. Implementation plan for Codex	16
Phase 0 - Pin and instrument	16
Phase 1 - Native baseline	16
Phase 2 - Event and scheduler hooks	16
Phase 3 - KV hook	16
Phase 4 - Closed-loop policy	17
Phase 5 - Optimize only after profiling	17
11.1 Suggested module layout	17
12. Experiment and decision framework	17
12.1 Comparison systems	17
12.2 Workload matrix	18
12.3 Primary metrics	18
12.4 Required ablations	18
12.5 Hardware go/no-go rule	19

13. What current harnesses actually do	19
13.1 When cache signals are emitted	20
13.2 Front-end priority pass-through	21
13.3 Cache signals versus priority signals	21
14. Latency prediction: what is realistic	22
14.1 Recommended predictor for the prototype	22
15. Risks, non-goals and design boundaries	23
15.1 Explicit non-goals for v1	23
16. Acceptance criteria for the first prototype	24
Appendix A. Controller API sketch	25
A.1 Lifecycle endpoints	25
A.2 Scheduler callback	25
A.3 KV commands	25
Appendix B. Policy pseudocode	25
Appendix C. Trace and metric schema	26
Appendix D. Source notes and references	27
Final recommendation	28

Reading guide

Sections 1-4 explain the idea. Sections 5-8 explain SGLang's queues and the three core hooks. Sections 9-12 give the exact controller design and experiment plan. The appendices preserve the harness survey, request schema, pseudocode and sources.

1. Executive summary

The proposed system is a software-defined, harness-aware resource controller placed between an agent harness and one SGLang worker. The harness supplies semantic facts that SGLang cannot reliably infer from token IDs alone: whether a user is waiting, whether the call is on the workflow's critical path, when a tool may finish, how likely a prefix is to be reused, how expensive recomputation would be, and which work may be delayed or cancelled. SGLang supplies live physical facts: queue length, prefix hits, KV residency, available and evictable blocks, measured restore time, current prefill/decode pressure, and recent execution latency. The controller turns both into three kinds of action: schedule work, manage KV state, and control admission/headroom.

The design starts in software because that is the fastest and most honest way to test the policy. If optimized software meets the SLO at high load with little sacrificed capacity, there is no need to harden the controller into silicon. Hardware becomes interesting only when measured fast-path costs - CPU reaction time, launch gaps, metadata scanning, coarse preemption or expensive resource switching - prevent the software design from meeting the same target efficiently.

Goal	Correct interpretation	What is required
Never miss deadlines	Guarantee accepted requests only; overload makes universal guarantees impossible.	Admission control, conservative p95/p99 estimates, headroom and a failure policy.
Always saturate GPU compute	Maximize useful work or goodput; decode may be memory-bandwidth-bound rather than compute-bound.	Phase-aware batching, chunked prefill and interference control.
No idle KV in HBM	Do not evict merely because a block is momentarily idle; keep it while its expected value exceeds its opportunity cost.	Reuse prediction, recompute cost, tier latency and memory pressure.
Use external storage maximally	Use it selectively. External reads consume time, bandwidth and money.	Place KV in the cheapest tier that can still satisfy the next-use deadline.

Recommended north-star metric

Maximize useful agent requests meeting their latency target per GPU-hour, while enforcing fairness and a background-throughput floor. Raw GPU utilization alone is not the objective.

2. Why harness knowledge changes the problem

To the inference engine, two requests may look nearly identical: both contain 80,000 input tokens and request up to 400 output tokens. To the harness, they may be completely different. One may be a live customer reply that must resume after a database call; the other may be an optional background branch. Without semantic hints, SGLang can optimize only from physical evidence such as arrival time, token counts, prefix locality and free KV capacity. Those are valuable but incomplete.

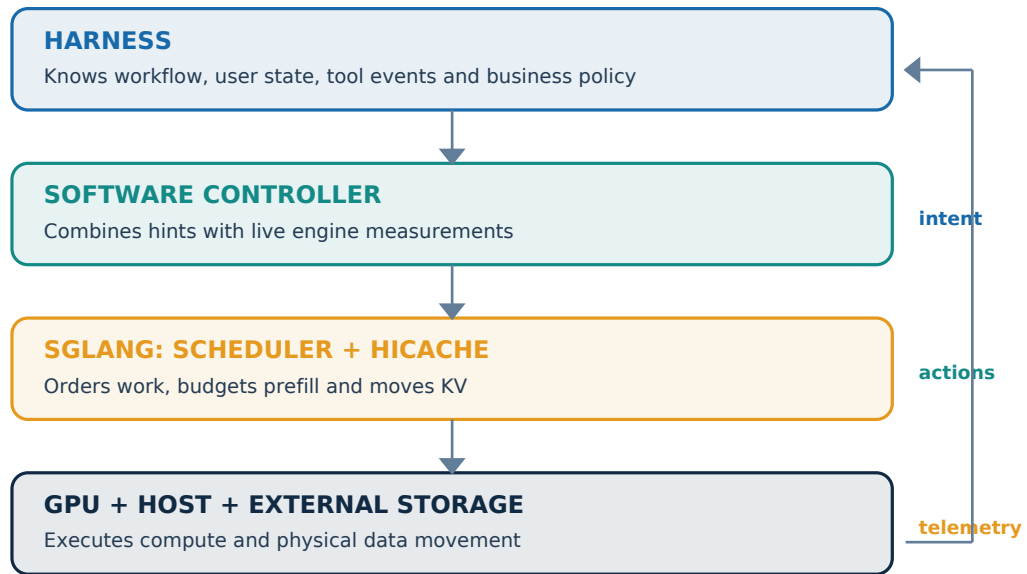


Figure 1. Software decides policy; SGLang and the device enforce physical actions.

2.1 Knowledge the harness already has

Knowledge	Example	Why SGLang cannot infer it reliably
Business importance	Customer escalation is more valuable than a nightly report.	Token shape does not encode organizational value.
User state	A person is actively waiting for the next token.	The same prompt can be interactive or asynchronous.
Workflow phase	Planning, tool wait, resumed synthesis, verification or background branch.	The model request is only one step in a larger graph.
Critical path	Three downstream steps are blocked on this result.	The backend sees one call, not dependent calls.
Future event	A database call is expected to finish in 30 seconds.	No model request exists yet for SGLang to inspect.
Optionality	A speculative branch can be cancelled without harming the answer.	The inference engine treats admitted work as required.
Reuse meaning	This session prefix will almost certainly be used after the tool returns.	Recent access is only a weak proxy for future reuse.
Data policy	KV may remain in host RAM but not shared external storage.	Privacy and retention rules live above the engine.

2.2 Knowledge SGLang already has

- The tokenized input, current sequence length, maximum requested output and sampling constraints.
- Arrival order, waiting requests, running decode requests, and whether structured-output grammar is ready.
- Prefix matches in the radix tree and how many prompt tokens can be reused.
- Available, actively used and evictable KV tokens; current host/L3 cache state when HiCache is enabled.
- Per-batch prefill token budgets, chunk limits, maximum running requests and memory conservativeness.
- Observed queue latency, TTFT, decode throughput, cache hit rate, transfer speed and transfer latency through metrics.

The controller's job

It should not decide what the agent says or which tool it is authorized to call. It decides when the remaining inference runs, where reusable state lives, and how much capacity may be consumed inside software-defined rules.

3. The signal contract: what the harness should communicate

The controller needs a stable, versioned contract. Hints are not commands: they describe intent and uncertainty. The controller validates them, combines them with measurements, and may refuse or downgrade a request when the requested SLO is infeasible.

3.1 Core signal groups

Group	Fields	Backend use
Identity and continuity	request_id, session_id, conversation_id, prefix_id, branch_id, tenant_id	Join tool events to cached state; recognize reuse families; enforce isolation.
Urgency and value	priority_class, user_waiting, deadline, hard/soft SLO, critical_path, value class	Queue ordering, protected capacity, preemption and admission.
Readiness	phase, ready_now, next_ready_eta, uncertainty, tool_started/completed/delayed	Proactive prefetch, timed headroom and event-triggered wake-up.
Work estimate	input tokens, output p50/p95, remaining calls, expected interarrival, model/LoRA	Capacity prediction, batching and memory reservation.
Cache value	reuse probability, recompute tokens/cost, shared-prefix ID, next-use window	Retention, eviction score and prefetch priority.
Placement policy	allowed tiers, minimum confidentiality, max retention, share scope	Prevent unsafe L2/L3 placement and cross-tenant reuse.
Control permissions	preemptible, cancelable, deferrable, degrade/model-switch allowed	Load shedding without violating application semantics.

3.2 Minimal first-version schema

```
{
  "schema_version": "agent-hints/v1",
  "request_id": "req-9921",
  "session_id": "support-417",
  "phase": "tool_wait",
  "priority_class": "critical",
  "user_waiting": true,
  "critical_path": true,
  "next_ready_eta_ms": 30000,
  "eta_uncertainty_ms": 2000,
  "slo": {
    "tool_completion_to_first_token_ms": 100,
    "hard": true
  },
  "workload": {
    "output_tokens_p50": 150,
    "output_tokens_p95": 400,
    "remaining_llm_calls_p50": 2
  },
  "cache": {
    "prefix_id": "support-417-turn-8",
    "reuse_probability": 0.98,
    "recompute_tokens": 80000,
    "allowed_tiers": ["hbm", "host", "external"],
    "share_scope": "session"
  },
  "control": {
    "preemptible": false,
    "cancelable": false,
    "deferrable": false
  }
}
```

3.3 Signals must include uncertainty

An ETA without uncertainty invites brittle scheduling. A tool whose median latency is 30 seconds but whose early tail reaches 28 seconds cannot be protected by starting a prefetch at 29.98 seconds. Use a distribution or at least an uncertainty interval. Likewise, output length should be a percentile range, not a single guessed number.

Bad hint	Better hint	Reason
tool returns in 30 s	p05=28 s, p50=30 s, p95=36 s	The earliest plausible return determines hard-SLO preparation.
output=200 tokens	p50=150, p95=400	Tail output controls memory and decode occupancy.
priority=10	class=critical, deadline=100 ms, user_waiting=true	A number alone does not explain urgency or feasibility.
cache=true	reuse=0.98, next-use window, recompute=80k tokens	The engine needs value and timing, not a binary preference.

4. What the backend can do with semantic knowledge

4.1 Deadline-aware admission and scheduling

- Estimate whether the request can meet its SLO under current load before promising it.
- Order accepted work by deadline risk, value and fairness rather than arrival time alone.
- Reserve only the headroom needed for the predicted critical window.
- Reject, defer, route, downgrade or shorten optional work when capacity is infeasible.

4.2 Proactive KV lifecycle management

- Keep near-term, expensive-to-recompute prefixes in HBM.
- Demote long-waiting state to host memory or external storage.
- Prefetch before a predicted resume instead of discovering a miss after arrival.
- Evict low-value unique tails before shared or high-reuse prefixes.

4.3 Better batching and prefix locality

- Group compatible requests that share a prefix when doing so does not endanger deadlines.
- Use large prefill chunks under quiet conditions and shrink them near urgent return windows.
- Mix compute-heavy prefill with memory-heavy decode only when interference measurements support it.

4.4 Work elimination and load shedding

- Cancel speculative branches after another branch resolves the question.
- Delay non-critical summaries or switch permitted work to a smaller model.
- Ask the harness to compact, filter or deduplicate tool results before expensive prefill.

4.5 Fairness, isolation and observability

- Age low-priority work so it cannot starve indefinitely.
- Honor tenant/cache-sharing boundaries and retention limits.
- Explain each action: which hint and measurement caused an admission, demotion or throttle decision.

Do not optimize the wrong thing

A decode-heavy batch can be limited by memory bandwidth while tensor cores are not fully occupied. The right target is goodput under SLOs, not permanent 100% compute utilization.

5. How one SGLang worker schedules requests

The description below is a conceptual map of the current Python SGLang runtime, grounded in its public documentation and source. Exact internal names can change across releases, so the prototype should pin a tested SGLang commit.

5.1 Request path

- 1 The API/tokenizer layer validates and tokenizes the request, attaches fields such as request ID, priority, session/cache identifiers and sampling parameters, then sends a request object to the scheduler process.
- 2 The scheduler receives new requests and places runnable ones in a waiting queue. Requests needing grammar preparation or other prerequisites may wait in a separate readiness path.
- 3 At each iteration, the scheduler processes the previous batch's results, removes finished or aborted work, frees or caches KV state, and updates the running decode batch.
- 4 It orders the waiting queue according to the active policy and computes prefix matches when the policy/cache requires them.
- 5 A prefill-admission component scans that order and adds requests while several budgets permit: KV capacity, maximum prefill tokens, chunk size, running-request limits and backend-specific constraints.
- 6 The scheduler chooses a prefill, decode or mixed batch; allocates/locks the necessary KV pages; constructs GPU inputs and launches execution.

- 7 Generated tokens and metrics are processed and streamed. Unfinished requests stay in the running batch; retracted work returns to a waiting path; then the loop repeats.

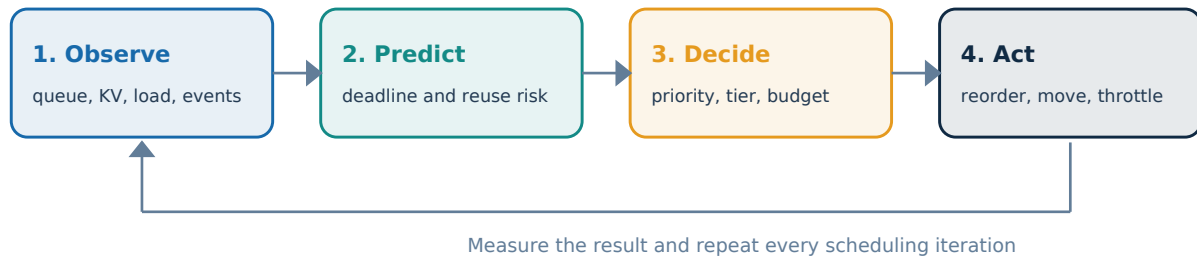


Figure 2. The scheduler is a repeated feedback loop, not a one-time sort.

5.2 The important queues and states

Structure/state	Meaning	Controller relevance
waiting queue	Requests eligible or nearly eligible for a new prefill/admission decision.	Reorder, age, or temporarily block background work.
running batch	Requests already in decode or otherwise active across iterations.	Protect interactive decode; choose safe preemption/retraction victims.
grammar/readiness wait	Requests waiting for structured-output grammar or preprocessing.	A high priority value cannot bypass a missing dependency.
chunked request	A long prompt whose prefill spans multiple scheduling iterations.	Chunk size defines how soon urgent work can intervene.
radix/HiRadix tree	Prefix metadata connecting token paths to KV pages in one or more tiers.	Reuse identity, cache matching, pinning and eviction.
KV pools	Device, host and optional external capacity with available/used/evictable state.	Physical feasibility of admission and restore.

5.3 Decisions the scheduler makes

Decision	Inputs it uses today	Semantic information missing without the harness
Queue order	Arrival, configured policy, prefix match, priority field, routing key and output bound.	True deadline, user waiting, critical path and business value.
Prefill admission	Uncached input length, max output estimate, free/evictable KV, prefill/chunk budgets.	Whether work is deferrable or whether later calls depend on it.
Decode continuation	Active sequences, KV availability and batch constraints.	Which response is directly blocking a person or workflow.
Retraction/preemption	Memory pressure, configured policy, length and priority difference.	Cost of losing progress and whether recomputation is acceptable.
Prefix reuse	Exact token-prefix match plus cache key/salt boundaries.	Expected next use and semantic ownership.
KV eviction	Recency/frequency/priority policy and lock status.	Tool return time, reuse probability and privacy tier policy.

5.4 Native policy and capacity mechanisms

SGLang currently exposes request policies including FCFS, longest-prefix-match, DFS-weight, longest-output-first, random, priority and routing-key modes. It also exposes priority scheduling, an optional direction for lower numeric values, a preemption threshold, a priority-aware retraction policy, maximum prefill tokens, schedule conservativeness, mixed chunking and a tunable number of continuous decode steps. These are powerful actuators, but most are configured globally and react to requests already visible to SGLang. [1][2][3]

Mechanism	What it provides	Limitation for this proposal
priority request field	A caller can attach an integer priority; SGLang's OpenAI-compatible schema and engine API expose it.	Priority is not a deadline, readiness forecast or cache-placement policy.
priority scheduling/preemption	Higher-priority work can be ordered first and may preempt when the configured difference is large enough.	Preemption occurs at safe software boundaries; it cannot promise immediate kernel interruption.
chunked prefill	Long prefills are broken into smaller scheduling units.	Smaller chunks improve responsiveness but can reduce throughput; size is not natively driven by future tool ETA.
prefix-aware policies	LPM/DFS-style policies increase KV reuse.	Locality may conflict with deadlines; semantic value is not known.
Radix eviction policy	LRU, LFU, segmented-LRU or priority-based eviction choices.	A general eviction rule is not a per-session next-use reservation.

6. What HiCache provides - and what it does not

HiCache extends SGLang's radix-prefix cache across three conceptual tiers: L1 GPU HBM, L2 host memory, and an optional L3 external/distributed backend. Its documented workflow includes local matching, prefetch and write-back. It supports asynchronous control/data paths, configurable write policies and L3 backends. This is the correct foundation for the KV part of the controller. [4][5]

Capability	Present in HiCache foundation?	Needed extension
GPU/host/external tiers	Yes.	Expose per-prefix placement intent and policy constraints.
Prefix matching	Yes, via HiRadixTree metadata.	Map harness prefix/session IDs to safe internal cache identities.
Prefetch on a request/cache miss	Yes.	Allow prefetch before the resumed inference request arrives.
Write-through/selective/write-back behavior	Yes, configurable.	Choose or override behavior per workload class/prefix where safe.
Automatic demotion because an agent entered tool wait	Not natively from harness semantics.	A tool-wait event that triggers backup/demotion after a policy decision.
Pin in host, forbid L3	Tiers exist; the semantic policy is not a standard request contract.	Host pin/lease and allowed-tier enforcement.
Deadline-aware restore	Timeout and fetch policies exist, but not the proposed agent deadline contract.	A prefetch deadline/priority queue driven by predicted next use.

A useful distinction
 Prompt-cache markers from hosted-model harnesses usually describe reusable prompt boundaries. SGLang KV control requires exact token-prefix identity and physical residency operations. A glue layer may translate useful identity, but a provider's cache_control object is not automatically a HiCache command.

7. The five tool-wait actions, one by one

Consider session support-417: an 80,000-token prefix, a database call expected near 30 seconds, a hard target of first token within 100 ms after tool completion, 0.98 reuse probability, and current high GPU pressure. The following table separates native machinery from the controller patch.

Desired action	How SGLang can help today	What the controller must add
1. Do not leave all 80k tokens in HBM for 30 s	Radix/HiCache can retain, evict and back up prefix KV as memory pressure changes.	On tool_started, score the prefix and issue a safe demotion: copy to host, confirm completion, then release eligible HBM pages. Do not evict if near-term reuse value exceeds pressure.
2. Keep KV in host rather than L3	HiCache has host and optional external tiers.	Add an allowed-tier/host-lease rule: <code>pin_host(prefix, until)</code> . Eviction may move other prefixes first; this prefix must not spill to disallowed storage.
3. Prefetch 15-20 ms before return	HiCache prefetches when a new request finds lower-tier hits.	Add a timer/event API that can prefetch by prefix ID before the resumed request exists. Compute time from measured p95 restore/setup plus uncertainty and safety margin.
4. Reduce background prefill near return	Chunked prefill, maximum prefill tokens, priority and preemption already provide coarse levers.	Add a temporary scheduling profile: shrink background token budget, prevent new large chunks and preserve a small request/KV headroom lease for the critical window.
5. Put resumed request first	Native priority fields and priority scheduling can reorder ready requests; threshold-based preemption exists.	Translate the tool-completion event into an immediate high-priority request, add deadline ordering/aging, and verify that the current chunk/preemption boundary can meet the SLO.

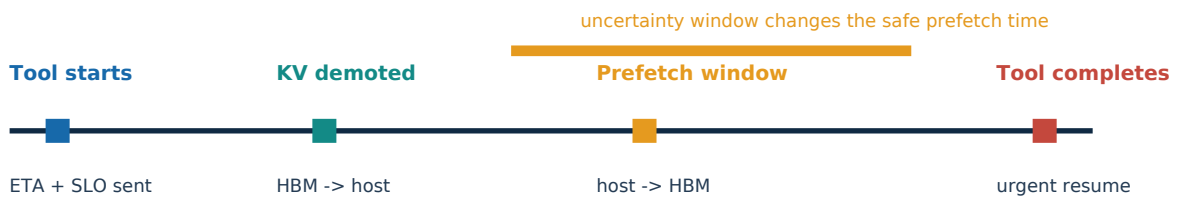


Figure 3. Tool-wait lifecycle. The safe preparation window depends on uncertainty, not only the median ETA.

7.1 Correct prefetch calculation

For an event with predicted ready time T , define R as measured p95 restore time, S as measured p95 scheduler/setup time, M as safety margin, and U as early-return uncertainty. A conservative hard-SLO preparation time is:

```
prefetch_start = predicted_ready_time - early_uncertainty - restore_p95
                - setup_p95 - safety_margin
```

Example: $T=30,000$ ms, $U=2,000$ ms, $R=8$ ms, $S=4$ ms, $M=4$ ms
start near 27,984 ms, not 29,980 ms.

That earlier prefetch means the KV may occupy HBM for about two seconds. The controller can instead choose event-triggered restore if $\text{restore_p99} + \text{setup_p99} + \text{queue_delay_p99}$ remains safely below 100 ms. The correct choice is an optimization over memory opportunity cost and deadline risk.

7.2 Enforcement state machine

State	Entry event	Controller action	Exit
ACTIVE	Inference running	Protect required KV; collect actual token/latency data.	tool_started or request_done
TOOL_WAIT_HOT	Short or uncertain wait	Keep HBM or begin host backup; no L3 spill if forbidden.	demotion timer or tool_completed
TOOL_WAIT_COLD	Long wait and host copy safe	Release HBM after verified host residency; schedule prefetch timer.	prefetch window or tool_completed
PREFETCHING	Timer or early completion	Raise KV transfer priority; reduce background prefill if needed.	resident or timeout
READY_URGENT	tool_completed	Admit first if feasible; otherwise invoke explicit fallback.	first token or rejection
DONE	Request/session completes	Release leases and eventually evict state per retention policy.	terminal

8. The three core hooks for a single worker

The first prototype should resist adding a large plugin surface. Three hooks are enough to test the idea end to end.

Hook 1 - Request intent and lifecycle events

A small front-door API receives normalized hints and asynchronous events from the harness. It owns validation, authentication, tenant boundaries, timestamps and idempotency. It writes controller state; it does not touch GPU memory directly.

Operation	Purpose	Example
register_request	Create intent and SLO state before or with inference.	priority, deadline, output p95, cache identity
tool_started	Mark a session not runnable and supply ETA distribution.	session, prefix, tool type, p05/p50/p95
tool_eta_updated	Move timers and leases when the tool is late/early.	new ETA and uncertainty
tool_completed	Wake the session immediately; cancel obsolete timers.	result-ready timestamp
cancel/branch_resolved	Remove optional work and release its reservations.	branch ID and reason

Hook 2 - Scheduler policy/admission hook

This hook runs inside, or immediately next to, SGLang's scheduler process. It can directly influence the internal waiting order and current batch construction, but only through safe invariants. The first version should return a small decision object rather than let an external Python service mutate lists concurrently.

```
SchedulerDecision {
  effective_priority: int
  deadline_ns: int | null
  admit: yes | no | defer
  max_prefill_tokens_this_round: int
  background_chunk_cap: int
  reserved_kv_tokens: int
  preemption_allowed: bool
  reason_code: string
}
```

- Re-score and reorder waiting requests before prefill selection.

- Shrink or restore the per-iteration prefill budget near a critical window.
- Keep a bounded KV/request headroom lease rather than permanently idling capacity.
- Choose a safe preemption/retraction victim using priority, progress-loss cost and fairness age.
- Refuse infeasible hard-SLO work before it disturbs admitted requests.

Hook 3 - KV lifecycle hook

This hook maps controller prefix identities to HiCache/HiRadix metadata and queues physical movement. Every operation is asynchronous and acknowledged; the controller must never free the only valid copy before a transfer completes.

```
backup_to_host(prefix_id, priority) -> operation_id
release_device_pages(prefix_id, only_if_host_valid=true)
pin_host(prefix_id, lease_until_ns, allowed_tiers=["host"])
prefetch_to_device(prefix_id, deadline_ns, priority) -> operation_id
evict(prefix_id, allowed_tiers, reason)
query_residency(prefix_id) -> {device_pages, host_pages, external_pages, inflight}
```

Why telemetry is not a fourth core hook

Metrics are the feedback channel used by all three hooks. Export queue, KV and transfer measurements, but keep the mutation surface limited to lifecycle events, scheduler decisions and KV operations.

9. Concrete software architecture

9.1 Components

Component	Responsibility	Deployment in v1
Harness adapter	Translate each harness's metadata/event model into agent-hints/v1.	In or beside the harness.
Intent store	Versioned per-request/session state, timers, leases and idempotency.	In-memory single process; append-only event log for replay.
Estimator	Rolling p50/p95/p99 for queue, prefill, decode and KV transfer costs.	Online empirical tables keyed by model and size bucket.
Policy engine	Admission, deadline-risk scoring, tier choice and headroom calculation.	Deterministic Python first; optimized C++ only after profiling.
SGLang scheduler adapter	Inject priority/deadline score and dynamic batch-budget decision.	Patch pinned SGLang scheduler commit.
HiCache adapter	Query residency; request backup, release, pin, prefetch and eviction.	Thin wrapper around existing cache controller/tree operations.
Telemetry/export	Prometheus metrics, trace spans and decision logs.	Existing SGLang metrics plus controller-specific counters.

9.2 Control cadence

- Event-driven: tool_started, ETA update, tool_completed, cancellation, request arrival and request completion.
- Scheduler-iteration: recompute queue scores and batch budgets from current memory/load state.
- Slow background loop: recalibrate latency/transfer models, expire leases and audit stale intent.

9.3 Recommended decision rule

Start with a legible deterministic score, not reinforcement learning. A request's scheduling score should rise with deadline risk, business class, user waiting and fairness age; it should fall when running it would consume excessive scarce KV or destroy more valuable cached state. Cache placement should compare expected recomputation/restore savings with the opportunity cost of each tier.

```
deadline_slack = deadline - now - predicted_start_cost_p95

schedule_score =
    deadline_risk(deadline_slack)
    + priority_weight
    + user_waiting_bonus
    + critical_path_bonus
    + fairness_age
    + prefix_reuse_savings
    - preemption_loss

cache_value =
    reuse_probability * avoided_recompute_ms
    - tier_restore_risk_ms
    - memory_opportunity_cost
```

Safety rule

Harness hints may be stale, wrong or adversarial. Clamp priority classes, authenticate tenant ownership, validate cache share scope, cap reservations, age every queue and expire every lease.

10. Detailed single-worker operating example

One GPU serves three concurrent workflows: a critical support session returning from tools, a normal coding agent with a 100,000-token prefill, and a low-priority reporting agent waiting on a 30-second database operation.

Workload	Harness facts	Controller behavior
Support agent	User waiting; hard 100 ms resume-to-first-token SLO; 80k-token prefix; high reuse.	Protect or prefetch KV, reserve just-in-time headroom, admit first on completion.
Coding agent	Normal priority; long prefill; chunkable and deferrable for short windows.	Run large chunks while quiet; reduce chunk size near critical window; continue with remaining capacity.
Reporting agent	Low priority; long tool wait; not user-facing; reusable context.	Back up/demote KV; no protected decode capacity; age so it eventually progresses.

10.1 Timeline

- 1 At $t=0$, the support agent starts a database call. The adapter registers the ETA distribution, hard SLO and prefix identity.
- 2 The estimator measures current device-to-host/host-to-device transfer cost for the relevant KV size. The policy verifies that a valid host copy can be created and chooses a demotion time.
- 3 After the host backup is acknowledged, the KV hook releases eligible HBM pages. The coding prefill uses the freed capacity.
- 4 As the earliest plausible return approaches, the controller starts prefetch or establishes enough headroom for an event-triggered restore. It reduces new background-prefill chunks only for the necessary window.
- 5 If the database is late, the harness updates the ETA. The controller cancels or moves the reservation rather than wasting headroom indefinitely.
- 6 If completion arrives early, the controller cancels the timer, immediately prioritizes the transfer and tests deadline feasibility from current measured state.

- 7 When the resumed request enters SGLang, the scheduler hook gives it the highest bounded effective score. It starts at the next safe scheduling boundary.
- 8 After the first token, the controller can relax the prefill throttle while continuing to protect an inter-token-latency target if configured.

10.2 Possible failure outcomes

Failure	Detection	Required behavior
Tool returns before safe window	tool_completed arrives while KV is host-only	Immediate high-priority restore; use reserved queue/KV headroom; record prediction miss.
Prefetch misses deadline	KV not resident by latest-start time	Stop starting background chunks; choose documented fallback; do not silently claim guarantee.
GPU genuinely saturated	Predicted earliest start already exceeds deadline	Reject/defer/degrade before admission or violate lower-value soft work by policy.
Host pool full	No capacity for backup/pin	Select lower-value host victim, use allowed external tier, or keep HBM; never discard sole copy.
Hint stale or session cancelled	generation/lease version mismatch	Ignore stale operation, release reservation and record reason.
Priority abuse	Untrusted caller requests critical class	Map through server-side policy and quotas; never trust raw numeric priority.

11. Implementation plan for Codex

Phase 0 - Pin and instrument

- Pin an exact SGLang commit and record GPU, CUDA, model, tensor-parallel and HiCache configuration.
- Enable request-time and cache metrics; add trace IDs linking harness events, controller decisions, scheduler iterations and KV operations.
- Measure p50/p95/p99 host restore, external restore, queue/setup, prefill-chunk and decode-step latency by size bucket.

Phase 1 - Native baseline

- Run vanilla SGLang with normal queue/cache settings.
- Run the best native configuration: priority enabled, bounded priorities, priority-aware retraction, chunked prefill, tuned maximum prefill tokens and HiCache host tier.
- Confirm what the public API actually preserves by logging the parsed request object and scheduler request priority/cache fields.

Phase 2 - Event and scheduler hooks

- Implement agent-hints/v1 validation and the intent store.
- Add tool_started/updated/completed/cancel lifecycle events.
- Patch the scheduler at one narrow pre-batch point to consume a decision snapshot: queue score, admission result and current prefill budget.
- Start with deadline-aware ordering and just-in-time background-prefill throttling; do not modify KV yet.

Phase 3 - KV hook

- Add prefix-to-HiCache identity mapping with strict tenant/session scope.
- Implement acknowledged backup-to-host, release-after-copy, host lease and prefetch-to-device.

- Make operations idempotent and versioned; survive early completion, delay and cancellation races.

Phase 4 - Closed-loop policy

- Use rolling measured percentiles instead of fixed assumed delays.
- Add admission control for hard SLOs and fairness aging for background work.
- Add realistic and oracle predictors to separate prediction error from controller overhead.

Phase 5 - Optimize only after profiling

- Profile Python control overhead and synchronization. Move only the measured critical path to C++ or an in-process extension.
- Preserve a software fallback and feature flags for every hook.
- Do not begin FPGA/RTL work until the best software version exposes a repeatable residual bottleneck.

11.1 Suggested module layout

```

controller/
  schema.py           # agent-hints/v1 validation
  events.py           # idempotent lifecycle event handling
  intent_store.py     # per-session state, leases, generations
  estimator.py        # empirical p50/p95/p99 latency tables
  admission.py        # hard-SLO feasibility
  policy.py           # queue score, budgets, tier choice
  timers.py           # prefetch/headroom windows
  telemetry.py        # metrics and decision log
adapters/
  harness_adapter.py  # frontend/harness -> normalized hints
  sglang_scheduler.py # narrow scheduler decision snapshot hook
  sglang_hicache.py   # acknowledged KV operations
tests/
  race_cases/         # early/late/cancel/tool-complete races
  trace_replay/       # deterministic workload replay

```

12. Experiment and decision framework

12.1 Comparison systems

System	Purpose	What is allowed
A. Vanilla SGLang	Current baseline.	Default/native scheduling and cache behavior.
B. Best native software	Strongest no-patch baseline.	Priority, chunked prefill, HiCache and tuned static settings.
C. Harness-aware controller	Prove semantic hints and three hooks.	Event-driven deadline, dynamic budget and proactive KV policy.
D. Oracle-hint controller	Upper bound for perfect future knowledge.	Replay exact tool completion and output length; not deployable.
E. Fast-path replay	Estimate hardware only after measurement.	Replace only measured controller/launch delay; preserve bandwidth and compute limits.

12.2 Workload matrix

Dimension	Values
Offered load	light; 60%; 75%; 85%; 95%; overload
Tool ETA accuracy	exact oracle; realistic distribution; early return; late return; no ETA
Prefix size	8k; 32k; 80k; model limit
Cache tier	HBM hit; host hit; external hit; total miss/recompute
Background interference	decode-heavy; long prefill; mixed; memory-transfer-heavy
Urgent concurrency	one critical resume; burst of critical resumes; conflicting hard SLOs
Failure/race	cancel during copy; ETA change; host full; transfer timeout; stale event

12.3 Primary metrics

Metric	Why it matters
Tool-completion-to-first-token	Direct measure of resumed agent responsiveness.
Hard-SLO attainment at p95/p99	Shows predictability, not just average improvement.
SLO-qualified requests per GPU-hour	Combines deadline value with capacity/cost.
Background throughput/fairness	Prevents the controller from winning by starving everything else.
Reserved/headroom fraction	Measures capacity sacrificed to guarantee urgency.
HBM/host/external residency time	Reveals whether placement matches next-use value.
KV bytes moved and transfer overlap	Shows data-movement cost and contention.
Prompt tokens recomputed	Captures failures to preserve valuable state.
Scheduler/controller CPU time and launch gaps	Identifies the residual hardware case.
Prediction calibration	Checks whether p95 predictions are actually met about 95% of the time.

12.4 Required ablations

- Remove deadline hints but keep priority.
- Remove ETA/uncertainty but keep tool events.
- Disable proactive KV prefetch while keeping scheduler control.
- Disable dynamic prefill throttling while keeping priority ordering.
- Use LRU-only cache placement versus semantic cache value.
- Compare realistic predictor, exact oracle and intentionally noisy predictor.

12.5 Hardware go/no-go rule

Observed result	Conclusion
Best software meets SLO at high load with little headroom	Keep policy in software; new hardware is not justified.
Software meets SLO only by permanently reserving large capacity	Finer/faster resource control may reclaim goodput; investigate hardware.
Misses come from ETA errors	Improve harness prediction or use safer admission; faster hardware alone does not solve it.
Misses come from no physical compute/bandwidth capacity	Scale or reject work; a controller only redistributes scarcity.
Misses come from measured CPU/launch/metadata delays or coarse safe points	Prototype the smallest relevant fast path in C++/FPGA/firmware and apply measured latency to real traces.

Strongest eventual hardware claim

Not 'hardware can meet the deadline,' but: hardware meets the same deadline at materially higher offered load, with less reserved capacity and lower cost than the best measured software design.

13. What current harnesses actually do

Most coding/agent harnesses do not independently label each request 'important, therefore cache it.' Their cache behavior is usually structural: keep stable system instructions, tools and conversation prefixes byte-consistent; add provider-specific cache breakpoints; reuse a session/cache key; or enable a configured retention mode. That is useful evidence that harness-originated signals exist, but it is not yet the same as the semantic SLO/KV contract proposed here.

Harness/platform	Documented cache/hint behavior	Decision basis	Relevance to SGLang
Deep Agents / LangChain	Anthropic prompt-caching middleware marks stable system/tool content and conversation history breakpoints; custom middleware/context are supported.	Content role and structural stability, not inferred business importance.	Good insertion point for the normalized hint adapter.
OpenCode	Prompt caching defaults to auto; protocol adapters place breakpoints at the last tool, last system part and latest user message where explicit caching exists.	Fixed structural rule; manual hints can override.	Cache hints need translation; extensions/provider options can carry extra metadata.
Qwen Code	Token caching is automatic for API-key flows; settings expose enableCacheControl, custom headers and extra request body fields.	Primarily provider/auth/config and repeated content; public docs do not describe semantic importance classification.	Straightforward place to inject SGLang extension fields through an adapter.
Pi	Global/environment cache-retention configuration and provider adapters; extensions and SDK/RPC make customization possible.	Configured retention/provider behavior, not per-request importance inference.	An extension can emit lifecycle and priority hints.
OpenClaw	Per-agent/model cacheRetention, stable/volatile system-prefix boundary, session/cache keys and provider-specific marker/key shaping.	Configuration, agent identity, stable prefix layout and provider capability.	Strong cache-identity precedent; still requires explicit SGLang mapping.

Harness/platform	Documented cache/hint behavior	Decision basis	Relevance to SGLang
Claude Code	Prompt caching is enabled in supported routes and can be disabled in some configurations; detailed internal breakpoint policy is not fully public.	Implementation/provider behavior, not a public semantic-priority classifier.	Use gateway/adaptor instrumentation; do not assume arbitrary priority passthrough.
Codex	Hosted/API prompt caching and service tiers exist at the platform boundary, but Codex's internal request-selection rules are not a public SGLang contract.	Provider/session behavior; no public evidence of per-request semantic KV placement for an external SGLang worker.	Use a controlled gateway or explicit harness integration rather than inference from observed cache traffic.
NeMo Agent Toolkit (NAT)	Prediction trie profiles call positions, predicts remaining calls/interarrival/output percentiles, computes latency sensitivity and injects <code>nnext.agent_hints</code> for Dynamo.	Learned workflow position plus optional annotations; this is the clearest native precedent.	Reuse the concept and map normalized fields into the single-worker controller.
NVIDIA Dynamo	Consumes routing hints and manages inference routing/cache concerns; it is closer to an inference platform/router than a coding harness.	Live infrastructure state plus application hints.	Conceptual precedent; multi-worker routing remains out of v1 scope.

Important finding

NAT already demonstrates that a harness can profile a workflow, predict per-call properties, compute latency sensitivity and inject backend hints. The proposal does not need to prove that hint generation is possible; it needs to prove that single-worker SGLang actions driven by those hints improve SLO-qualified goodput. [9]

13.1 When cache signals are emitted

In general, these systems do not first classify a request as important and then decide whether to cache it. When caching is enabled, they apply a structural/configured rule to each eligible model invocation; the provider decides whether the prefix is long enough and identical enough to produce a cache hit. A marker may therefore appear on every eligible request even though only some requests actually reuse cached work.

System	Typical emission rule	Per-request importance decision?
Deep Agents	When prompt-caching middleware is enabled, stable system/tool content and selected history boundaries are marked on eligible calls.	No. The default rule follows content role/stability.
OpenCode	Every request defaults to <code>cache=auto</code> unless opted out; explicit-cache providers receive selected breakpoints, while implicit-cache providers may receive no inline marker.	No. Automatic placement is a fixed structural policy; manual hints can override.
Qwen Code	API-key flows use automatic token caching when supported; configuration can enable cache control.	Public docs describe automatic cost optimization, not semantic importance classification.
Pi	A configured cache-retention mode is applied through the relevant provider adapter.	No documented built-in per-request importance classifier.
OpenClaw	Merged global/model/agent <code>cacheRetention</code> and provider capabilities shape each eligible request; stable-prefix boundaries and session keys are reused.	Usually configuration and session structure, not request importance.
NAT agent hints	Hints are injected when prediction-trie profiling and a Dynamo backend are configured; per-call-position predictions can replace static values.	Yes, it can compute latency sensitivity from workflow data, but this is an agent hint rather than a generic cache marker.

13.2 Front-end priority pass-through

A useful prototype question is not whether the harness uses SGLang's exact field name. It is whether a front-end value can survive the harness and appear at an adapter boundary, where glue code can map it to SGLang's priority field and the richer controller contract. Public documentation supports the following cautious classification:

Harness/platform	Can a priority-like value survive?	Recommended prototype path
Deep Agents / LangChain	Yes with explicit runtime context, state or middleware; no universal built-in SGLang-priority field.	Read trusted front-end metadata in middleware, map policy server-side, and emit agent-hints/v1.
OpenCode	Provider-specific request options and extensions make it possible; arbitrary SGLang priority is not a documented portable default.	Use a controlled provider/extension adapter and inspect the prepared outgoing body.
Qwen Code	Yes: settings expose custom headers and extra_body for OpenAI-compatible requests.	Put a bounded priority/hint object in extra_body, then validate it in the gateway.
Pi	Possible through extensions, SDK/RPC or custom provider code; no documented native semantic-priority field.	Implement one small extension that copies trusted front-end metadata into the normalized adapter.
OpenClaw	Per-agent/model parameters and transport shaping exist, but a portable user-supplied per-call priority is not the documented default.	Use an agent hook/transport adapter; keep business-to-numeric priority mapping on the server.
Claude Code	A gateway can observe and route traffic, but arbitrary front-end priority reflection is not a documented stable Claude Code contract.	Instrument a controlled gateway; do not rely on undocumented request internals.
Codex	OpenAI API requests support service tiers, but the Codex harness is not documented as a generic arbitrary-priority pass-through to an external SGLang backend.	Use an explicit integration or gateway; treat observed fields as version-specific until verified.
NeMo Agent Toolkit	Yes. Request metadata/middleware can carry values, and the profiler can natively generate latency-sensitivity and workflow hints for Dynamo.	Use NAT as the reference adapter and map nvext.agent_hints into the normalized schema.
Dynamo	Yes as a hint consumer/router, not as the originating coding harness.	For v1, borrow its hint concepts but send decisions to one SGLang worker.

Trust boundary

Passing a value through is not the same as authorizing it. A user-controlled priority=critical must be mapped through server-side identity, quota and policy before SGLang sees a bounded numeric priority.

13.3 Cache signals versus priority signals

Signal	What it normally means	What it does not mean
cache_control / cache breakpoint	Content before this boundary is intended to be reusable under provider rules.	This request is high priority or its KV must stay in HBM.
prompt/session cache key	Route or account repeated prefixes consistently; separate cache lineages.	The prefix is valuable enough to pin indefinitely.
cache retention	Ask for none/short/long provider retention where supported.	Choose host versus external storage on a self-hosted engine.
priority/service tier	Request faster or preferred processing within a specific API/service contract.	A portable cross-harness deadline guarantee.
agent hints	Describe workflow/interarrival/output/latency sensitivity for backend optimization.	An unconditional command that overrides resource feasibility or safety.

14. Latency prediction: what is realistic

There is no universal model that can look at an arbitrary incoming SGLang request and always predict its exact first-token time after processing starts. TTFT depends on the model, hardware, token length, prefix hit, batch composition, queue state, chunking, kernels, parallelism and interference - many of which change between scheduling iterations.

Approach	Strength	Limitation
Empirical online tables	Simple, calibrated to the exact deployment; ideal for v1.	Needs enough samples and careful bucketing.
Regression/GBDT/random forest	Can model nonlinear effects of input length, batch tokens and cache hits.	Must be retrained when software, model or hardware changes.
High-fidelity simulator such as Vidur	Published work reports low aggregate prediction error after profiling and predicts TTFT/TBT/throughput.	Not a drop-in exact online oracle for SGLang's current live queue; validation was on profiled configurations.
Oracle trace replay	Separates controller potential from prediction error.	Research upper bound only; cannot be deployed.
Formal schedulability model	Can offer conservative guarantees with bounded execution costs and admission.	Bounds may be pessimistic and require disciplined preemption/chunk assumptions.

14.1 Recommended predictor for the prototype

- Key by model/quantization, GPU type, TP size, prefix-hit tier, prefill token bucket, decode batch bucket and active transfer state.
- Maintain exponentially weighted p50/p95/p99 for queue delay, setup, prefill chunk, decode step and tier restore.
- Predict a distribution and check calibration; do not optimize only mean absolute error.
- Fall back to conservative static bounds for unseen buckets.
- Record predicted-versus-actual at every request so the controller improves without hidden assumptions.

Use the harness for the future, not for GPU physics

The harness is best at predicting when a call will occur and what role it plays. The engine is best at predicting how long that call will take under the current batch/cache state. Combine the two distributions.

15. Risks, non-goals and design boundaries

Risk	Mitigation
Deadline guarantees become marketing language	Guarantee only admitted hard-SLO requests under documented assumptions; expose rejection and miss reasons.
Priority starvation	Bound classes, add aging/quotas and maintain a background goodput floor.
Cache side channels or tenant leakage	Use tenant-scoped identities/salts, allowed-tier policy and no cross-tenant prefix reuse by default.
Stale events cause wrong movement	Version events, make operations idempotent, expire leases and reject old generations.
Copy storms increase latency	Token-bucket DMA/preload queues, coalesce duplicate prefix operations and include transfer contention in estimates.
Controller violates SGLang invariants	Return immutable decision snapshots at defined hooks; never mutate live queues from another thread.
Tight coupling to upstream changes	Pin a commit, isolate adapters, add scheduler/HiCache contract tests and rebase deliberately.
False hardware conclusion	Compare vanilla, best native software, harness-aware software and measured fast-path replay.

15.1 Explicit non-goals for v1

- No multi-worker routing, prefill/decode disaggregation or cross-GPU KV migration in the first prototype.
- No model-content or tool-authorization decisions in the controller.
- No arbitrary hardware partitioning claims that current GPUs cannot enforce.
- No reinforcement-learning policy before a deterministic baseline is understood.
- No claim that every provider cache marker maps directly to physical SGLang KV pages.

16. Acceptance criteria for the first prototype

Area	Minimum acceptance test
Correctness	No sole-copy KV loss; event operations are idempotent; cancellation and early/late tool races pass.
Deadline value	Meaningful reduction in p99 tool-completion-to-first-token and miss rate versus best native software at matched offered load.
Efficiency	Background throughput loss is bounded and reported; controller does not win by leaving permanent unused capacity.
Cache value	Fewer recomputed prompt tokens and/or lower high-value restore latency with measured tier-residency evidence.
Prediction	p95 cost predictions are calibrated within an agreed tolerance on held-out traces.
Operability	Every admission, throttle, demotion, prefetch and rejection has a reason code and trace correlation.
Decision	Results clearly indicate keep-in-software, improve prediction, scale capacity, or prototype a specific hardware fast path.

Recommended first demonstration

A critical support agent resumes within its 100 ms target after a tool event while a 100k-token coding prefill continues making progress and an unrelated long-wait KV prefix is safely demoted. Compare against both vanilla SGLang and the best native priority/chunked-prefill/HiCache configuration.

Appendix A. Controller API sketch

A.1 Lifecycle endpoints

```
POST /v1/controller/requests
POST /v1/controller/events/tool-started
POST /v1/controller/events/tool-eta-updated
POST /v1/controller/events/tool-completed
POST /v1/controller/events/cancelled
GET /v1/controller/sessions/{session_id}
GET /v1/controller/prefixes/{prefix_id}/residency
```

A.2 Scheduler callback

```
def before_batch_build(snapshot: SchedulerSnapshot) -> SchedulerDecision:
    # snapshot: waiting/running summaries, KV counts, current load, now
    # decision: ordered request IDs, admission flags, temporary budgets
    # requirement: bounded runtime and no blocking I/O
    ...
```

A.3 KV commands

```
def request_backup(prefix_id, destination="host", priority=...) -> OpId: ...
def request_prefetch(prefix_id, deadline_ns, priority=...) -> OpId: ...
def release_device(prefix_id, require_valid_copy=True) -> Ack: ...
def set_lease(prefix_id, tier="host", until_ns=..., generation=...) -> Ack: ...
def cancel_operation(op_id, generation=...) -> Ack: ...
```

Appendix B. Policy pseudocode

```
on tool_started(event):
    validate_owner_and_generation(event)
    state = intent_store.update(event)
    costs = estimator.for_prefix(state.prefix_id)
    plan = choose_kv_tier(state, costs, live_memory_pressure())
    if plan.backup_to_host:
        op = kv.backup_to_host(state.prefix_id, plan.priority)
        after_ack(op):
            if state.generation_is_current():
                kv.release_device_pages(state.prefix_id, only_if_host_valid=True)
    timers.schedule(plan.prefetch_start, state.generation)

on tool_completed(event):
    state = intent_store.update(event)
    timers.cancel_older(state.session_id, state.generation)
    if not kv.is_device_resident(state.prefix_id):
        kv.prefetch_to_device(state.prefix_id,
                             deadline=state.first_token_deadline,
                             priority=CRITICAL)
    scheduler.publish_urgent_resume(state.request_id, state.deadline)

before_batch_build(snapshot):
    decisions = []
    for req in snapshot.waiting:
        pred = estimator.predict_start_and_ttft(req, snapshot)
        slack = req.deadline - snapshot.now - pred.p95
        score = policy.score(req, slack, snapshot)
        decisions.append((req.id, score))
    budget = policy.prefill_budget(snapshot, imminent_deadlines())
    return SchedulerDecision(order=sort(decisions),
                             max_prefill_tokens=budget,
                             admissions=admission.check_all(...))
```

Appendix C. Trace and metric schema

Field/metric	Type	Meaning
controller_event_total{type}	counter	Validated lifecycle events by type.
controller_decision_seconds	histogram	Software decision time.
controller_deadline_slack_ms	histogram	Predicted slack at admission and dispatch.
controller_admission_total{result,reason}	counter	Admit/defer/reject/degrade decisions.
controller_prefill_budget_tokens	gauge	Dynamic prefill cap applied this iteration.
controller_headroom_kv_tokens	gauge	Temporary KV reservation.
controller_kv_operation_seconds{op,tier}	histogram	Backup/prefetch/release latency.
controller_kv_residency_tokens{tier,class}	gauge	KV allocation by tier and semantic class.
controller_prediction_error_ms{component}	histogram	Actual minus predicted queue/prefill/restore/decode cost.
agent_tool_resume_ttft_ms{class}	histogram	Tool completion to first streamed token.
agent_slo_total{class,result}	counter	Met/missed/rejected hard and soft SLOs.
agent_background_goodput	gauge	Completed background tokens/tasks per unit time.

Appendix D. Source notes and references

Ref	Source and relevance
[1]	SGLang Server Arguments Scheduling policies, priority scheduling/preemption, retraction, chunking, radix eviction and scheduler controls. https://docs.sglang.io/docs/advanced_features/server_arguments
[2]	SGLang OpenAI protocol source SGLang extension fields include request priority, extra_key and cache_salt. https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/entrypoints/openai/protocol.py
[3]	SGLang schedule policy source Queue ordering, prefix matching and scheduling-policy implementation. https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/schedule_policy.py
[4]	SGLang HiCache design L1 GPU, L2 host and L3 external storage; match, prefetch and write-back workflow. https://docs.sglang.io/docs/advanced_features/hicache_design
[5]	SGLang HiCache controller source Host cache controller, prefetch capacity and transfer machinery. https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/cache_controller.py
[6]	SGLang schedule batch source Request and batch data structures; CPU-side ScheduleBatch to GPU-side ForwardBatch flow. https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/schedule_batch.py
[7]	SGLang scheduler source Event loop, waiting/running state and batch-selection implementation. https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/managers/scheduler.py
[8]	SGLang metrics collector Queue, KV availability/use, transfer and throughput metrics. https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/observability/metrics_collector.py
[9]	NeMo Agent Toolkit profiler Prediction trie, output/interarrival/remaining-call predictions, latency sensitivity and Dynamo agent hints. https://docs.nvidia.com/nemo/agent-toolkit/1.5/improve-workflows/profiler.html
[10]	NeMo Agent Toolkit request metadata Request attributes and workflow-accessible request metadata. https://docs.nvidia.com/nemo/agent-toolkit/latest/api/nat/runtime/user_metadata/index.html
[11]	LangChain Anthropic prompt-caching middleware Structural caching of stable system prompts, tools and conversational prefixes. https://docs.langchain.com/oss/python/integrations/middleware/anthropic
[12]	OpenCode LLM cache design Automatic and manual cache hints, provider-specific translation and breakpoint placement. https://github.com/anomalyco/opencode/blob/dev/packages/llm/README.md
[13]	Qwen Code token caching Automatic token caching behavior and reported cache usage. https://qwenlm.github.io/qwen-code-docs/en/users/features/token-caching/
[14]	Qwen Code configuration Cache control, custom headers and extra request-body configuration. https://qwenlm.github.io/qwen-code-docs/en/users/configuration/settings/
[15]	OpenClaw prompt caching Cache retention, stable-prefix boundaries, provider behavior and cache keys. https://github.com/openclaw/openclaw/blob/main/docs/reference/prompt-caching.md
[16]	Pi coding agent README Extensibility, providers, cache usage reporting and cache-retention configuration. https://github.com/badlogic/pi-mono/tree/main/packages/coding-agent
[17]	Vidur: LLM inference simulator Profile-based prediction of latency, throughput and deployment performance. https://www.microsoft.com/en-us/research/publication/vidur-a-large-scale-simulation-framework-for-llm-inference/

Ref	Source and relevance
[18]	Sarathi-Serve (OSDI 2024) Prefill/decode characteristics and throughput-latency tradeoffs under chunked scheduling. https://www.usenix.org/conference/osdi24/presentation/agrawal
[19]	Prompt Cache paper Schema-defined reusable prompt modules and KV reuse for lower TTFT. https://arxiv.org/abs/2311.04934

Source snapshot note: software interfaces and repository internals change quickly. Pin the exact SGLang and harness versions used by the experiment and re-check every hook before implementation. Public documentation does not expose every private internal policy of hosted coding agents; where evidence is incomplete, this report says so rather than inferring intent from observed network traffic.

Final recommendation

Build the best software controller first
Implement three narrow hooks - lifecycle events, scheduler/admission decisions, and KV lifecycle operations - around one SGLang worker. Use real harness hints, measured p95/p99 costs and explicit admission control. Demonstrate improved SLO-qualified goodput against the best native software baseline. Only then identify a measured fast path worth hardening into hardware.

The architectural opportunity is not a smarter queue in isolation. It is a closed loop in which the harness describes future workflow value, SGLang reports present physical constraints, and a bounded controller continually chooses the most valuable feasible work while preserving reusable state.